Lab Assignment-1

School of Engineering and Technology
Course Name & Code: Operating System Lab ENCA252
Program Name: BCA (AI & DS) (Research)

Submission Guidelines

General Requirements

- **Submission Deadline:** Assignments must be submitted within one week of the assignment's release date.
- **Submission Platform:** All assignments are to be submitted via the Learning Management System (LMS).
- **GitHub Link:** You must provide a link to your GitHub repository with your submission.
- **Individual Submission:** Assignments are to be completed and submitted by each individual student.
- **Formatting:** All assignments must adhere to the specific format shared in class.

Evaluation

- **Total Marks:** This assignment is worth a total of 5 marks.
- **Evaluation Metrics:** Assignments will be evaluated based on the following criteria:
    - **Originality:** The uniqueness and independent thought demonstrated in the work.
    - **Correctness:** The accuracy and validity of the solutions or content.

Problem Title:

Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

Problem Statement

CPU Scheduling is a core function of an operating system that determines the order in which processes access the CPU. In this assignment, students will implement FCFS and SJF scheduling algorithms and evaluate their performance using standard metrics.

Learning Objectives

• Understand CPU scheduling concepts
• Implement FCFS and SJF algorithms
• Calculate CT, TAT, WT
• Analyze system performance

Tools/Technology Used:

• Python 3.x
• Built-in Python libraries: os, multiprocessing, time
• Linux OS (optional for simulation)

Assignment Tasks:

<u>Task 1: Process Creation & Input Handling</u>

Objective:

To understand how processes are represented in an operating system and how input data is structured for scheduling.

Description:

In an operating system, each process has attributes such as Process ID (PID), Arrival Time (AT), and Burst Time (BT). This task focuses on creating a process model using a class and taking user input.

Sub-Tasks:

1. Create a class Process with attributes:
    - Process ID (PID)
    - Arrival Time (AT)

    - Burst Time (BT)

2. Accept input for at least 4–5 processes from the user.
3. Store processes in a list or array.
4. Display the input in tabular format.

Expected Outcome:

- Students will understand process structure.
- Proper input handling and data storage.

<u>Task 2: FCFS Scheduling Implementation</u>

Objective:

To implement the First Come First Serve (FCFS) scheduling algorithm.

Description:

FCFS is the simplest scheduling algorithm where the process that arrives first gets executed first. It is non-preemptive in nature.

Sub-Tasks:

1. Sort processes based on Arrival Time.
2. Calculate:
    - **Completion Time (CT)**
    - **Turnaround Time (TAT) = CT – AT**
    - **Waiting Time (WT) = TAT – BT**
3. Handle CPU idle condition (if no process has arrived yet).
4. Display results in a table.

Expected Outcome:

- Understanding of FCFS working.
- Ability to compute scheduling parameters.

<u>Task 3: SJF Scheduling (Non-Preemptive)</u>

Objective:

To implement Shortest Job First scheduling.

Description:

SJF selects the process with the smallest burst time from the ready queue. It improves average waiting time but requires careful selection.

Sub-Tasks:

1. From available processes, pick the one with the shortest burst time.
2. Ensure only arrived processes are considered.
3. Calculate:
    - Completion Time (CT)
    - Turnaround Time (TAT)
    - Waiting Time (WT)
4. Continue until all processes are executed.

Expected Outcome:

- Better understanding of optimization in scheduling.
- Comparison with FCFS.

Task 4: Gantt Chart Representation

Objective:

To visualize process execution sequence.

Description:

A Gantt chart shows how processes are executed over time.

Sub-Tasks:

1. Draw execution order for:
    - FCFS
    - SJF
2. Represent time intervals clearly.

3. Label processes on timeline.

Expected Outcome:

- Visual understanding of scheduling.
- Improved conceptual clarity.

<u>Task 5: Performance Analysis & Comparison</u>

Objective:

To evaluate and compare scheduling algorithms.

Description:

Performance is measured using average waiting time and turnaround time.

Sub-Tasks:

1. Calculate:
    - Average Waiting Time
    - Average Turnaround Time
2. Compare FCFS and SJF results.
3. Answer:
    - Which algorithm is better?
    - Why?

Expected Outcome:

- Analytical thinking.
- Understanding trade-offs between algorithms.

Student Instructions:

Submit all Python source code files and screenshots of output.
Maintain proper code structure and add meaningful comments.

Submit a summary report in .pdf or .docx format explaining your code and outputs.
Late submissions will lead to grade penalties as per academic policy.

Rubrics for Evaluation

Criterion	Weight	Excellent (5)	Good (4)	Fair (3) or Poor (2 or less)
Implementation & Execution	40%	Correct simulation of all components	Minor logic or output issues	Significant errors or missing outputs
Code Quality & Documentation	30%	Well-commented and structured code	Mostly readable, few comments	Poor formatting or no documentation
Output Accuracy	20%	All outputs correct and validated	Minor output mismatches	Incomplete or incorrect output
Report & Submission	10%	Submitted on time, clear report	Minor issues or delay	Late or no report